

NOTAS A MANO SOBRE
ANÁLISIS DE COMPLEJIDAD
COMPUTACIONAL

Segunda edición

NOTAS A MANO SOBRE ANÁLISIS DE COMPLEJIDAD COMPUTACIONAL

Segunda edición

Autores:

Carlos Eduardo Orozco Garcés

César Jesús Pardo Calvache

Mauro Callejas Cuervo

Capítulo 4

Ejercicios propuestos

Soluciones: Análisis de complejidad temporal y espacial

1. Asignación y retorno de variables enteras

Se realiza una asignación de dos variables enteras y un retorno inmediato. Cada operación tiene tiempo constante y no depende del tamaño de entrada.

$$T(n) = 1$$

$$S(n) = 1$$

2. Retorno basado en una condición compuesta

La expresión lógica se evalúa completamente en tiempo constante. No se realizan ciclos ni llamadas adicionales.

$$T(n) = 1$$

$$S(n) = 1$$

3. Validación simple (positivo, negativo o cero)

Se evalúan una o dos condiciones simples. El número de operaciones es constante y no depende de la entrada.

$$T(n) = 1$$

$$S(n) = 1$$

4. Suma de elementos en un arreglo

Se recorre el arreglo una única vez con un ciclo *for*, realizando una suma por cada elemento.

$$T(n) = n$$

$$S(n) = 1$$

5. Obtener los múltiplos de un número arbitrario

Se imprime cada múltiplo de un valor dado hasta n , incrementando de forma lineal en pasos de tamaño fijo.

$$T(n) = \left\lfloor \frac{n}{m} \right\rfloor = O(n)$$

$$S(n) = 1$$

6. División logarítmica

El valor se divide por 2 en cada iteración de un ciclo *while*, hasta que el número sea menor o igual que 1.

$$T(n) = \log_2(n)$$

$$S(n) = 1$$

7. Determinar si un número es primo

Se verifican posibles divisores desde 2 hasta $\sqrt{n}$. El mejor caso ocurre cuando se encuentra un divisor temprano, y el peor si es primo.

$$T_{\text{mejor}}(n) = 1$$

$$T_{\text{peor}}(n) = \sqrt{n}$$

$$S(n) = 1$$

8. Imprimir números pares en un arreglo

Se generan todas las combinaciones de pares de índices (i, j) con $i < j$. La cantidad de iteraciones es cuadrática.

$$T(n) = O(n^2)$$

$$S(n) = 1$$

9. Imprimir números pares iguales en un arreglo

Es similar al anterior, pero con una verificación de igualdad adicional por cada par.

$$T(n) = O(n^2)$$

$$S(n) = 1$$

10. Multiplicación de matrices

Tres ciclos anidados recorren cada celda de la matriz resultado, multiplicando y sumando valores correspondientes.

$$T(n) = n^3$$

$$S(n) = n^2$$

11. Contar divisiones sucesivas

Un ciclo *while* divide el número por 2 sucesivamente hasta llegar a 1. La cantidad de iteraciones es logarítmica.

$$T(n) = \log_2(n)$$

$$S(n) = 1$$

12. Ciclo con incremento en saltos de dos unidades

El ciclo recorre la mitad de los valores hasta n y ejecuta una función con tiempo $\log_2(n)$ en cada iteración.

$$T(n) = \frac{n}{2} \cdot \log_2(n) = O(n \log n)$$

$$S(n) = n$$

13. Llamada directa a función primalidad

La función simplemente invoca otra que verifica si un número es primo, y por tanto hereda su complejidad.

$$T(n) = \sqrt{n}$$

$$S(n) = 1$$

14. Ciclos arbitrarios con funciones anidadas

La función contiene:

- Un ciclo *for* que ejecuta $foo1(i)$
- Un ciclo *while* que ejecuta $foo2()$
- Una llamada única a $foo3()$

Dadas las complejidades $T_{foo1}(n) = 3n! + 5n$, $T_{foo2}(n) = 2n^2 + 3$, y $T_{foo3}(n) = n + 5$, se obtiene:

$$T(n) = O(n \cdot n!)$$

$$S(n) = n$$

15. Funciones arbitrarias (invocación directa)

Se invocan secuencialmente dos funciones cuyas complejidades son $\sqrt{n}$ y $\log_2(n)$, respectivamente.

$$T(n) = \sqrt{n} + \log_2(n)$$

$$S(n) = n$$